

Algoritmi e Strutture Dati – Parte A – 16/06/2021

Esercizio A1 – Punti ≥ 8

Si consideri la seguente funzione di complessità:

$$T(n) = \begin{cases} 16T(n/4) + f(n) \end{cases}$$

Calcolare la complessità di $T(n)$ per i seguenti valori di $f(n)$, utilizzando il Master Theorem (normale o esteso); scrivere "N/A" (non applicabile) se non è possibile utilizzare nessuna delle due versioni; nel caso si utilizzi il Caso 3 del Master Theorem versione estesa, si dimostri anche la condizione aggiuntiva.

1. $f(n) = n$
2. $f(n) = n \log n$
3. $f(n) = n^2$
4. $f(n) = n^2 + \log n$
5. $f(n) = n^2 / \log n$
6. $f(n) = n^2 \log n$
7. $f(n) = n^3$
8. $f(n) = n^3 \log n$

Esercizio A2 – Punti ≥ 10

Scrivere un algoritmo

boolean findPair(**int**[] A_1 , **int**[] A_2 , **int** n , **int** k)

che prenda in input due vettori ordinati di interi contenenti n elementi ciascuno e un intero k , e restituisca **true** se esistono due valori $a_1 \in A_1$ e $a_2 \in A_2$ tali che $a_1 + a_2 = k$, **false** altrimenti.

Discutere informalmente la correttezza dell'algoritmo e calcolare la sua complessità computazionale.

Per esempio, se $A_1 = [1, 5, 8, 10]$, $A_2 = [31, 37, 43, 52]$ e $k = 51$, l'algoritmo deve restituire **true** perché esiste la coppia $8 + 43$; se invece $k = 6$, l'algoritmo deve restituire **false** (in quanto $1 + 5$ non possono essere considerati in quanto appartengono allo stesso vettore).

Esercizio A3 – Punti ≥ 12

Un albero binario *completo* è un albero binario tale che:

- Tutte le foglie hanno profondità h o $h - 1$
- Tutti i nodi a livello h sono "accatastati" a sinistra
- Tutti i nodi interni hanno grado 2, eccetto al più uno

Scrivere un algoritmo

boolean isComplete(TREE T)

che prenda in input un albero binario T e restituisca **true** se l'albero è completo, **false** altrimenti.

Discutere informalmente la correttezza dell'algoritmo e calcolare la sua complessità computazionale.

Dei due alberi seguenti, quello di sinistra è completo, quello di destra no. Vi ricordo che abbiamo introdotto gli alberi binari completi parlando della struttura dati *heap*.

